Code Jam 2013 - Round 1C

Analysis: Consonants

Solving the small

It cannot be simpler than trying each possible substring given the name. For a given substring, we just check if there exists **n** consecutive consonants. If it is true, we count this substring into part of the **n**-value. There are O($\mathbf{L}^2$) substrings, and it takes O(**L**) time to check for at least **n** consecutive consonants. In total each case takes O($\mathbf{L}^3$) time to solve, which is acceptable to solve the small input. This approach is, of course, not fast enough to solve the large input.

Improving the naive algorithm

In fact we can skip the linear time checking for all possible substrings. Here we assume the index is zero based. Suppose we start from the **i**-th character. We also have **c** that starts as zero. When we iterate up to the **j**-th character, if it is a consonant, we increase **c** by 1, otherwise reset it to zero. Actually **c** is the number of consecutive consonants that starts after the **i**-th character and ends at the **j**-th character. If we meet the first instance such that **c** ≥ **n**, we can conclude that every substring which starts at the **i**-th character and ends at the **k**-th character, where **k** ≥ **j**, is the desired substring. Then we know that we can add **L** - **j** to the answer, and proceed to the next starting character. This algorithm runs in O($\mathbf{L}^2$) time, which is still not sufficient in solving the large input. But the concept of computing **c** is the key to solve the problem completely.

Further improving

Let us extend the definition of **c** to every character, call it $\mathbf{c_i}$: the number of consecutive consonants that ends at the **i**-th character. For example, suppose the string is **quartz**, then $\mathbf{c_0}$ = 1, $\mathbf{c_2}$ = 0, and $\mathbf{c_5}$ = 3. We can use similar approach mentioned in the last section to compute every $\mathbf{c_i}$ in O(**L**) time. Also define a pair (**x**, **y**) to be the substring that starts at the **x**-th character and ends at the **y**-th character.

Knowing from the previous section, if we know that $\mathbf{c_i}$ ≥ **n**, then we know that substrings (**i** - $\mathbf{c_i}$ + **p**, **i** + **q**), where 1 ≤ **p** ≤ $\mathbf{c_i}$ - **n** + 1 and 0 ≤ **q** ≤ **L** - **i** - 1, are the desired substrings. It implies that there are ($\mathbf{c_i}$ - **n** + 1) × (**L** - **i**) substrings. If you proceed like this, you missed some substrings. Consider the string **axb** with **n** = 1. We see that $\mathbf{c_1}$ = 1 but we only count 2 substrings, namely **x** and **xb**. We miss the **prefix** options, namely **ax** and **axb**. It looks like we can consider the substrings (**p**, **i** + **q**), where 0 ≤ **p** ≤ **i** - **n** + 1 and 0 ≤ **q** ≤ **L** - **i** - 1. Unfortunately, in this case we may count certain substrings multiple times. Consider the string **xaxb** with **n** = 1, where we count **xax** and **xaxb** twice since $\mathbf{c_0}$ = $\mathbf{c_2}$ = 1.

To correctly count the substrings, we need to choose the appropriate range of **p**. In fact, we just need one more value: the last **j** < **i** such that $\mathbf{c_j}$ ≥ **n**. Let **r** = **j** - **n** + 2 if there is such **j**, or **r** = 0 otherwise. Then we have the right set of substrings (**p**, **i** + **q**), where **r** ≤ **p** ≤ **i** - **n** + 1 and 0 ≤ **q** ≤ **L** - **i** - 1. In fact, **r** means the longest possible prefix so that (**r**, **i** - **n**) contains at most **n** - 1 consecutive consonants and therefore we avoid repeated counting. Hence for each $\mathbf{c_i}$ ≥ **n** we count (**i** - **n** - **r** + 2) × (**L** - **i**). Summing up we have the answer. **r** is updated whenever we see that $\mathbf{c_i}$ ≥ **n** before iterating the next position. Therefore it takes constant time to update the value. Overall the running time is O(**L**), which is enough to solve the large input.

Despite the complications, the algorithm is extremely simple. The following is a sample solution:

```
def Solve(s, n):
  L = len(s)
  cnt, r, c = 0, 0, 0
  for i in range(L):
    c = c + 1 if s[i] not in "aeiou" else 0
    if c >= n:
      cnt += (i - n - r + 2) * (L - i)
      r = i - n + 2
  return cnt
```

Code Jam 2013 - Round 1C

Analysis: Pogo

The small dataset

The small dataset did not require finding the optimal solution, instead accepting any solution that solved the problem within 500 moves. This allowed for a variety of approaches. One such approach is as follows: note that with two subsequent jumps in two opposite directions you can move one unit in a chosen direction. By a sequence of at most 200 such pairs (so in total at most 400 moves) you can reach any point with $|X|, |Y| \leq 100$.

There were also other approaches possible, for instance a shortest-path search, if one could prove or guess that it's OK to limit the search space somehow. It turns out that one can actually just search for a path within the points with $|x|, |y| \leq 100$, we will see why in the next section. Thus, Dijkstra's algorithm or just breadth-first search will provide a short enough path to the target.

The large dataset

For the large dataset, we not only need to return the best possible solution, but also deal with more distant targets, so neither of the approaches above will work. We will begin with a few easy observations:

- First, if we want to reach the target in **N** moves, we have to have $1 + 2 + ... + \mathbf{N} \geq |\mathbf{X}| + |\mathbf{Y}|$.
- Moreover, if we want to reach the target in **N** moves, the parity of the numbers $1 + 2 + ... + \mathbf{N}$ and $|\mathbf{X}| + |\mathbf{Y}|$ has to be the same. This is because the parity of the sum of the lengths of jumps we make in the North-South direction has to match the parity of $|\mathbf{Y}|$, and the sum of lengths of West-East jumps has to match the parity of $|\mathbf{X}|$.

It turns out that if **N** satisfies these two conditions, it is possible to reach (X, Y) with **N** jumps.

Let's consider a point (**X**, **Y**), and any **N** satisfying the two conditions above. For the sake of brevity, assume $|\mathbf{X}| \geq |\mathbf{Y}|$, and $\mathbf{X} \geq 0$ (it's easy to provide symmetric arguments for the other four cases). In this case, we will assume the last move was East. This means the first **N**-1 moves have to reach (**X**-**N**, **Y**). We will proceed recursively, so we just have to prove that (**X**-**N**, **Y**) and **N**-1 satisfy the conditions above.

For **N** = 1 or 2, it's easy to enumerate all the possible **X** and **Y**. For **N** = 1, there are four possibilities, and in each case our strategy produces the correct move. For **N** = 2, if we assume **X** is positive and greater than $|\mathbf{Y}|$, the only possibilities are (3, 0), (2, 1), (2, -1) and (1, 0); after the move they turn to (1, 0), (0, 1), (0, -1) and (-1, 0) — all of which satify the conditions for **N** = 1.

For larger **N**, the parity condition is trivial — both considered parities stay the same if **N** was even, and both change if **N** is odd. For the inequality condition, if $\mathbf{N} \leq \mathbf{X}$, both sides just decrease by **N**, so the only interesting case is if $\mathbf{X} < \mathbf{N}$. However, in this case, we move to (**X** - **N**, **Y**), and the sum of absolute values is $\mathbf{N} - \mathbf{X} + |\mathbf{Y}| \leq \mathbf{N} - \mathbf{X} + \mathbf{X} = \mathbf{N} \leq 1 + ... + \mathbf{N} - 1$. Thus, the conditions are satisfied after one move, and we can continue with our strategy.

This logic again translates to simple code:

```
def Solve(x, y):
  N = 0
```

```
sum = 0
while sum < abs(x) + abs(y) or (sum + x + y) % 2 == 1:
  N += 1
  sum += N
result = ""
while N > 0:
  if abs(x) > abs(y):
    if x > 0:
      result += 'E'
      x -= N
    else:
      result += 'W'
      x += N
  else:
    if y > 0:
      result += 'N'
      y -= N
    else:
      result += 'S'
      y += N
  N -= 1
return result.reversed()
```

Code Jam 2013 - Round 1C

Analysis: The Great Wall

The small input

Despite the very long statement, solving the small input wasn't actually that hard. Still, the long statement scared many contestants off, which is probably why we saw the first submission only after half an hour of the contest, and relatively few submissions to the problem in general. With at most 10 tribes, at most 10 attacks and all the attacks happening on a short section of the Wall, we can just simulate all that happens. Let's look at it a bit more carefully.

Since **delta_p** is limited by 10, a tribe attacks at most 10 times, and the initial attack is between -100 and 100, all the attacks will occur between -200 and 200. Thus, we can afford to remember the height of the wall at each interesting point. This brings us to the first trick of this problem — what are the points we should be interested in?

Note that since the edges of attacked areas are always integers, the height of the wall in each open interval (*x, x+1*) for integral *x* is always constant. Moreover, the height at the integral points is never lower than at any of the two neighboring open intervals, since any attack that affects any of these intervals will also affect the integral point next to it. As $\mathbf{w_i} < \mathbf{e_i}$, any attack always affects at least one whole interval, and so the success of the attack depends only on the height of the wall in the intervals, and not on the edges. Thus, it is enough to keep information about the height of the wall in points of the form **x** + 0.5 for integral **x**. There are 400 such points to consider in the small input, and the height of each is initially zero.

There are a 100 attacks to consider. We can begin by generating all of them explicitly (noting the beginning and end point, day and strength for each of them), and sorting them by time of occurrence. For each day on which at least one attack occurs, we first check for each attack whether it succeeds (by examining the wall height at each attacked interval). Afterwards, for all attacks we go over all affected intervals and increase the height of the wall if necessary. Note that it is important to increase the wall height only after checking *all* the attacks that occur on a given day.

The large input

The numbers are much bigger for the large input. We can have 10^6 attacks, and they can range over an interval of length over 10^8. Let's analyse which parts of the previous approach will work, and which will not.

We can still generate all the attacks explicitly, and sort them by time. We probably need a more concise way to represent the Wall, though, and we surely need a faster way to check whether an attack succeeds and updating wall heights.

The problem of concise representation can be solved by noticing that since we have only 10^6 attacks, we will have around 10^6 interesting points. A sample way to take advantage of this it to "compress" all attack coordinates — sort all the coordinates that are beginnings or ends of attacks, and consider as interesting only the points in the middles of intervals of adjacent endpoints. We will end up with at most 2×10^6 points, and each will represent an interval such that the height of the wall on this interval is always the same. Using this tric to compress the attack coordinates, we can assume all attacks happen in a space of at most 2×10^6 points. We can rename these points to be consecutive for convenience.

To attack the problem of checking attack success and updating the wall, we will need some variant of an interval tree. We will present two interval-tree based approaches below.

An interval tree is a tree, in which each node represents an interval [**m** x $2^{\mathbf{k}}$, **m** x $2^{\mathbf{k}}$] for some **m,k**. The parent of a node containing an interval **I** will be the node representing a twice longer interval containing **I** (so, if **I** is [**m** x $2^{\mathbf{k}}$, **m** x $2^{\mathbf{k}}$], the parent is [(**m** / 2) x $2^{\mathbf{k+1}}$, (**m**/2 + 1) x $2^{\mathbf{k+1}}$]). This is the common pattern for interval trees, the trick is in what to store in nodes.

High and low

In the first approach, we will try to answer the questions directly by the means of using a modified interval tree. We will store two values in each node — hi and lo. The "hi" value will be pretty standard, and will be defined so that the height of the wall at any given point is the maximum "hi" value of all the intervals containing this point. This can be updated in logarithmic time when any interval of the wall is attacked - we can split any interval into a logarithmic number of intervals represented by nodes, and update the hi value in each of them. This will allow us to update the wall height, and to figure out what the height of the wall at a given point is, each in logarithmic time. We still need a way to figure out whether an attack will succeed in logarithmic time, though.

We will use the "lo" values for that. For a given node **X** and a path to a leaf from **X** we can define the maximum "hi" value on this path as the "partial height" of the leaf node. This is what would be the height, if we disregarded all the nodes above **X** (in particular, "partial heights" measured from the root node are simply wall heights). We now define the "lo" value of **X** as the smallest partial height of a descendant of **X**. We need to see how this is useful, and how to update it in logarithmic time when updating the "hi" values.

Note that if we have a "lo" value for a node calculated, we can easily figure out the height of the lowest wall point in this interval - it's the maximum of the "lo" value of this node and the "hi" values of all the ancestors of this node. Thus, to figure out whether an attack will succeed on a general interval we split it into a logarithmic number of intervals represented by nodes, and figure out the lowest wall segment in each of these sub-intervals. If any of these is lower than the strength of the attack, it will succeed. This is logarithmic-squared as described, but it's easy to implement it to actually be logarithmic.

Now note that the "lo" values have a simple recursive definition - take the minimum of the "lo" values of the children, or the "hi" value of the node itself, whichever is higher. This means that when updating the "hi" value for a node, we only need to update the "lo" values for this node and its ancestors - meaning we can update "lo" values in logarithmic-squared time for each attack (and, again, it's simple to update them in logarithmic time).

Order by strength

Another approach that allows us to solve this problem with an interval tree is to order the attacks by strength, descending, and not by chronology. In this approach, for each point we know what was the earliest time at which it was attacked. Note that since we process from the strongest attack, any section of the wall that was attacked earlier by an attack we already processed is immune to attacks that come later and are processed later. Thus, to learn whether the attack is successful we need to find what's the latest attack time in the whole interval this attack covers; and subsequently we need to update the attack times to the minimum of the time that was stored so far, and the time of the currently processed attack.

This is called a min-max interval tree (we update with the minimum, and we query for the maximum). We encourage you to figure out what to store in the nodes to make this work in logarithmic time!